1 Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się z drzewami decyzyjnymi poprzez zbudowanie klasyfikatora, który będzie przewidywał, czy pacjent może doznać udaru na podstawie różnych parametrów wejściowych, takich jak płeć, wiek, schorzenia, przebyte choroby oraz palenie tytoniu. Ćwiczenie obejmuje również analizę i przygotowanie danych, implementację różnych metod opartych na drzewach decyzyjnych oraz ocenę ich wydajności, co pozwala na zgłębienie problematyki drzew decyzyjnych.

2 Treść zadania

Według Światowej Organizacji Zdrowia (WHO) udar jest drugą najczęstszą przyczyną zgonów na świecie i odpowiada za około 11% wszystkich zgonów. Zadanie polega na budowie klasyfikatora opartego na drzewach decyzyjnych do przewidywania, czy pacjent może doznać udaru na podstawie parametrów wejściowych, takich jak płeć, wiek, schorzenia i przebyte choroby oraz palenie tytoniu. Każdy wiersz danych zawiera istotne informacje o pacjencie.

3 Część teoretyczna

3.1 Drzewa decyzyjne

Drzewa decyzyjne są popularnym narzędziem w uczeniu maszynowym, które służy do klasyfikacji i regresji. Drzewo decyzyjne składa się z

- węzłów, czyli punktów w drzewie, w których dokonywany jest podział danych na podstawie określonej cechy, każdy węzeł reprezentuje test decyzyjny, który określa, jak dane powinny być podzielone w dalszej części drzewa,
- gałęzi, czyli połączeń między węzłami, które reprezentują wynik testu decyzyjnego w poprzednim węźle,
- liści, czyli końcowych węzłów w drzewie, które nie mają dalszych podziałów i reprezentują ostateczną decyzję lub klasę, do której zostaną przypisane dane, które dotarły do tego liścia.

W zadaniu wykananym na rzecz tego sprawozdania wykorzystałam dwa rodzaje drzew decyzyjnych – drzewa klasyfikacyjne i drzewa regresji.

3.2 Drzewa klasyfikacyjne

Drzewa klasyfikacyjne, jak sama nazwa wskazuje, są stosowane do klasyfikacji danych. Używają kryteriów takich jak entropia lub współczynnik Gini do wyboru najlepszego testu. Entropia mierzą nieuporządkowanie danych, a celem jest wybranie podziału, który zwiększy jednorodność danych. Na każdym etapie

budowy drzewa, dla każdego węzła, algorytm sprawdza wszystkie możliwe podziały dla wszystkich cech i wybiera ten podział, który daje najlepszy wynik według wybranego kryterium, np. jak bardzo podział zmniejsza nieuporządkowanie. Drzewo jest budowane do momentu osiągnięcia określonej głębokości `max_depth`.

3.3 Regresja

Drzewa regresji, jak nazwa wskazuje, są narzędziem uczenia maszynowego stosowanym do przewidywania wartości ciągłych. Ich struktura jest podobna do drzew klasyfikacyjnych, ale zamiast klas decyzji w liściach, drzewa regresji zawierają wartości liczbowe. Najpopularniejszym kryterium wyboru sposobu podziału danych w węźle jest błąd średniokwadratowy (MSE), który mierzy średnią kwadratową różnic między przewidywanymi wartościami a rzeczywistymi. Celem jest minimalizacja tego błedu, więc dla każdego potencjalnego podziału obliczane jest, jak bardzo zmniejsza on MSE i wybierany jest podział, który daje najlepszy wynik.

3.4 Oversampling

Oversampling polega na „rozmnażaniu" danych klasy mniejszościowej w celu zrównoważenia liczebnośi klas i poprawy dokładności klasyfikacji. Problem niezbalansowanych danych, gdzie jedna klasa jest znacznie liczniejsza od drugiej, może być rozwiązywany za pomocą techniki oversamplingu.

3.5 Lasy losowe (Random Forest)

Lasy losowe to zespół drzew decyzyjnych, gdzie każde drzewo jest trenowane na losowo wybranym podzbiorze danych i z użyciem losowo wybranych atrybutów. Zamiast polegać na pojedynczym drzewie, las losowy buduje wiele drzew i łączy ich przewidywania, co zmniejsza ryzyko nadmiernego dopasowania i poprawia ogólną wydajność. Lasy losowe mogą poprawić jakość predykcji i zmniejszają ryzyko przeuczenia (overfitting). Każde drzewo w lesie jest trenowane na losowym podzbiorze danych treningowych, zwykle tego samego rozmiaru co oryginalny zbiór, ale z powtórzeniami. Podczas budowy każdego węzła w drzewie, algorytm losowo wybiera podzbiór cech do rozważenia dla podziału. Ostateczne przewidywanie jest średnią (dla regresji) lub głosowaniem większościowym (dla klasyfikacji) wyników z wszystkich drzew w lesie.

3.6 Gradient Boosting

Gradient Boosting to metoda, w której drzewa decyzyjne są budowane sekwencyjnie, tak aby każde kolejne drzewo korygowało błędy poprzednich drzew. Modele są dodawane jeden po drugim, gdzie każdy nowy model próbuje poprawić błędy modelu poprzedniego. Metoda zawiera parametr, który kontroluje, jak duży wpływ ma każdy nowy model na ostateczne przewidywanie. W każdym

kroku, przewidywania nowego modelu są mnożone przez ten parametr przed dodaniem do całościowego modelu, więc im mniejsza jego wartość, tym mniejszy wpływ ma każde nowe drzewo. Gradient Boosting pozwala na uzyskanie dokładniejszych modeli predykcyjnych.

4 Wykonanie ćwiczeń

4.1 Zadanie 1

Polecenie: Zaimportuj niezbędne moduły.

4.2 Zadanie 2

Polecenie: Wczytaj dane, dokonaj selekcji danych (każdy student tworzy inny zbiór danych, który składa się z wylosowanych wierszy danych oryginalnych).

Rozwiązanie:

```
data = pd.read_csv("healthcare-dataset-stroke-data.csv")
nr_gr = 3.8
r_k = 2025
np.random.seed(int(nr_gr*r_k))
q = data.shape
idx = np.random.permutation(q[0])
data = data.iloc[idx[0:5000],:]
data.head()
```

4.3 Zadanie 3

Polecenie: Scharakteryzuj dane.

Rozwiązanie:

```
data = pd.read_csv("healthcare-dataset-stroke-data.csv")
nr_gr = 3.8
r_k = 2025
np.random.seed(int(nr_gr*r_k))
q = data.shape
idx = np.random.permutation(q[0])
data = data.iloc[idx[0:5000],:]
data.head()
```

1 to 5 of 5 entries Filter

index	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
1366	35737	Male	1.08	0	0	No	children	Urban	86.09	19.5	Unknown	0
772	72547	Male	61.0	0	0	Yes	Private	Rural	55.26	33.2	Unknown	0
316	72214	Male	61.0	0	0	Yes	Self-employed	Urban	69.15	27.7	formerly smoked	0
3450	60088	Male	49.0	1	0	Yes	Self-employed	Rural	92.26	33.1	formerly smoked	0
468	28645	Female	38.0	1	0	Yes	Private	Urban	196.01	28.1	never smoked	0

```
1 data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5000 entries, 1366 to 1795
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   id                 5000 non-null   int64
 1   gender             5000 non-null   object
 2   age                5000 non-null   float64
 3   hypertension       5000 non-null   int64
 4   heart_disease      5000 non-null   int64
 5   ever_married       5000 non-null   object
 6   work_type          5000 non-null   object
 7   Residence_type     5000 non-null   object
 8   avg_glucose_level  5000 non-null   float64
 9   bmi                4802 non-null   float64
 10  smoking_status     5000 non-null   object
 11  stroke             5000 non-null   int64
dtypes: float64(3), int64(4), object(5)
memory usage: 507.8+ KB
```

```
1 data.describe()
```

1 to 8 of 8 entries Filter

index	id	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5000.0	5000.0	5000.0	5000.0	5000.0	4802.0	5000.0
mean	36491.2682	43.300856	0.0982	0.0542	105.94533799999999	28.91547271970012	0.0488
std	21206.82889800009	22.58739231087872	0.29761464160047935	0.22643457007026133	45.11382810295501	7.838721925154448	0.21547121749577
min	67.0	0.08	0.0	0.0	55.12	10.3	0.0
25%	17660.75	25.0	0.0	0.0	77.19	23.6	0.0
50%	36898.5	45.0	0.0	0.0	91.82	28.1	0.0
75%	54770.75	61.0	0.0	0.0	113.8625	33.1	0.0
max	72940.0	82.0	1.0	1.0	267.76	97.6	1.0

4.4 Zadanie 4

Polecenie: Oczyszczanie danych.

Rozwiązanie: Usuń nieprzydatną kolumnę `id`.

```
[5]  1 data=data.drop('id', axis = 1)
```

```
[6]  1 data.describe()
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5000.000000	5000.000000	5000.000000	5000.000000	4802.000000	5000.000000
mean	43.300856	0.098200	0.054200	105.945338	28.915473	0.048800
std	22.587392	0.297615	0.226435	45.113828	7.838722	0.215471
min	0.080000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	25.000000	0.000000	0.000000	77.190000	23.600000	0.000000
50%	45.000000	0.000000	0.000000	91.820000	28.100000	0.000000
75%	61.000000	0.000000	0.000000	113.862500	33.100000	0.000000
max	82.000000	1.000000	1.000000	267.760000	97.600000	1.000000

Znajdź i ewentualnie usuń zduplikowane wiersze.

```
[7]  1 data.duplicated().sum()
np.int64(0)
```

Nie ma żadnych zduplikowanych wierszy.

Znajdź atrybuty z brakującymi wartościami.

```
1 data.isnull().sum()
```

	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	198
smoking_status	0
stroke	0

dtype: int64

Widać, że w niektórych wierszach brakuje wartości `bmi`.

Usuń wiersze z brakującymi wartościami.

```
[9]  1 data = data.dropna(axis = 0)
1 data.isnull().sum()
```

	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	0
smoking_status	0
stroke	0

dtype: int64

Usuwam te wiersze i widzę, że teraz wartość `bmi` jest niepusta dla każdego wiersza.

4.5 Zadanie 5

Polecenie: Implementacja drzew decyzyjnych w bibliotece `sklearn` nie akceptuje atrybutów nominalnych. Takie atrybuty należy zamienić na liczbowe.

Rozwiązanie: Utwórz nową tabelę z danymi.

```
[11]  data2 = pd.DataFrame()
```

Przepisz do tej tabeli wszystkie atrybuty liczbowe.

```
[12]  data2['age'] = data['age']
      data2['avg_glucose_level'] = data['avg_glucose_level']
      data2['heart_disease'] = data['heart_disease']
      data2['hypertension'] = data['hypertension']
      data2['bmi'] = data['bmi']
      data2['stroke'] = data['stroke']
```

(Z wcześniej wykonanego zadania 3. (`data.info()`) wiem, jakie typy mają jakie kolumny, więc mogę przypisać bezpośrednio te, które były liczbowe).

Jeśli atrybut nominalny ma tylko dwie wartości, zamień te wartości na 0 lub 1.

```
data.nunique()
```

	0
gender	3
age	104
hypertension	2
heart_disease	2
ever_married	2
work_type	5
Residence_type	2
avg_glucose_level	3779
bmi	414
smoking_status	4
stroke	2

dtype: int64

```
[14]  data['gender'].unique()
array(['Male', 'Female', 'Other'], dtype=object)

[15]  data['work_type'].unique()
array(['children', 'Private', 'Self-employed', 'Govt_job', 'Never_worked'],
      dtype=object)

[16]  data['Residence_type'].unique()
array(['Urban', 'Rural'], dtype=object)

[17]  data['smoking_status'].unique()
array(['Unknown', 'formerly smoked', 'never smoked', 'smokes'],
      dtype=object)

[18]  data2['ever_married'] = np.int64(data['ever_married']=='Yes')

[19]  data2['urban_residence'] = np.int64(data['Residence_type']=='Urban')
```

Sprawdzam, jakie wartości mają inne atrybuty, aby znaleźć te, które mają tylko dwie. Są to `ever_married` i `Residence_type`.

Jeśli atrybut nominalny ma k>2 wartości, utwórz dla niego k nowych atrybutów.

```
[20]  1 data2['work_type_Private'] = np.int64(data['work_type']=='Private')
      2 data2['work_type_Self_emp'] = np.int64(data['work_type']=='Self-employed')
      3 data2['work_type_Govt_job'] = np.int64(data['work_type']=='Govt_job')
      4 data2['work_type_Children'] = np.int64(data['work_type']=='children')
      5 data2['work_type_Never_worked'] = np.int64(data['work_type']=='Never_worked')

[21]  1 data2['smoking_status_unknown'] = np.int64(data['smoking_status']=='Unknown')
      2 data2['smoking_status_formerly_smoked'] = np.int64(data['smoking_status']=='formerly smoked')
      3 data2['smoking_status_never_smoked'] = np.int64(data['smoking_status']=='never smoked')
      4 data2['smoking_status_smoked'] = np.int64(data['smoking_status']=='smokes')

      1 data2['gender_female'] = np.int64(data['gender']=='Female')
      2 data2['gender_male'] = np.int64(data['gender']=='Male')
      3 data2['gender_other'] = np.int64(data['gender']=='Other')
```

Dla reszty atrybutów tworzę nowe.

Ostatecznie uzyskałam następującą postać moich danych.

```
[26]  1 data2.describe()
```

	age	avg_glucose_level	heart_disease	hypertension	bmi	stroke	ever_married	urban_residence	work_type_Private	work_type_Self_emp	work_type_Govt_job	work_type_Children	work_type_Never_worked
count	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000	4802.000000
mean	42.934862	105.113020	0.048771	0.092870	29.015473	0.042482	0.654519	0.508484	0.573470	0.158120	0.128072	0.139085	0.004373
std	22.530317	44.233023	0.217464	0.289956	7.559732	0.201799	0.475274	0.500036	0.494772	0.365058	0.334254	0.346089	0.065950
min	0.080000	55.120000	0.000000	0.000000	10.300000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	25.000000	78.990000	0.000000	0.000000	23.600000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	44.000000	91.620000	0.000000	0.000000	28.100000	0.000000	1.000000	[illegible]	1.000000	0.000000	0.000000	0.000000	0.000000
75%	60.000000	113.320000	0.000000	0.000000	33.100000	0.000000	1.000000	1.000000	1.000000	0.000000	0.000000	0.000000	0.000000
max	82.000000	267.760000	1.000000	1.000000	97.600000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

```
      1 data2.head()
```

_smp	work_type_Govt_job	work_type_Children	work_type_Never_worked	smoking_status_unknown	smoking_status_formerly_smoked	smoking_status_never_smoked	smoking_status_smoked	gender_female	gender_male	gender_other
0	0	1	0	1	0	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	0
1	0	0	0	0	1	0	0	0	1	0
1	0	0	0	0	1	0	0	0	1	0
0	0	0	0	0	0	1	0	1	0	0

4.6 Zadanie 6

Polecenie: Przygotuj dane do uczenia modelu. Utwórz tabele:
- X z przykładami (zawierającą wszystkie kolumny z wyjątkiem `stroke`) i
- y z etykietami (kolumna `stroke`).
Podziel dane na część treningową (80% danych) i testową (20%). Przeprowadź próbne uczenie modelu.

Rozwiązanie:

```
[25]  1 data2.columns[5]

      'stroke'

[26]  1 X = data2.drop(columns=data2.columns[5])  #strok w szóstej kolumnie
      2 y = data2.iloc[:, 5]
      3 X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2, random_state = 1)
      4 dt = DTC(criterion='entropy', max_depth=5, random_state=1)
      5 dt.fit(X_train, y_train)
      6 y_pred = dt.predict(X_test)
      7 print(accuracy_score(y_test, y_pred))

      0.963579604578564
```

Powyższy kod wykonuje podane w poleceniu wytyczne. Uzyskuję dobrą dokładność powyższej klasyfikacji.

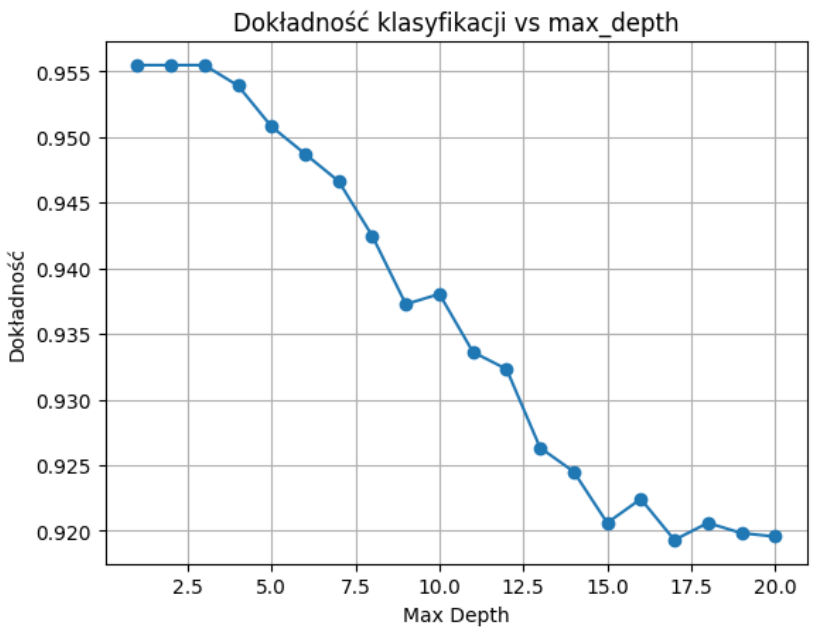
4.7 Zadanie 7

Polecenie: W procedurze kroswalidacji znajdź optymalną wartość `max_depth`. Pokaż wykres zależności dokładności klasyfikacji od `max_depth`. Odnotuj dokład-

ność modelu wyznaczoną w kroswalidacji.

Rozwiązanie:

```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

depths = range(1, 21)
cv_scores = []

for d in depths:
    dt = DTC(criterion='entropy', max_depth=d, random_state=1)
    scores = cross_val_score(dt, X_train, y_train, cv=5)
    cv_scores.append(scores.mean())

optimal_depth = depths[np.argmax(cv_scores)]
print(f'Optymalna wartość max_depth: {optimal_depth}')
print(f'Maksymalna dokładność w kroswalidacji: {max(cv_scores):.4f}')

plt.plot(depths, cv_scores, marker='o', linestyle='-')
plt.xlabel('Max Depth')
plt.ylabel('Dokładność')
plt.title('Dokładność klasyfikacji vs max_depth')
plt.grid()
plt.show()
```



Na wykresie widać, że optymalna głębokości to 1, 2 albo 3. Dokładne wartości dokładności przedstawiam poniżej – potwierdzają one wniosek wyciągnięty z wizualizacji.

```python
for d in range(1, 10):
    dt = DTC(criterion='entropy', max_depth=d, random_state=1)
    scores = cross_val_score(dt, X_train, y_train, cv=5)
    print(f'max_depth={d}, CV Accuracy: {scores.mean():.4f}')
```

```
max_depth=1, CV Accuracy: 0.9555
max_depth=2, CV Accuracy: 0.9555
max_depth=3, CV Accuracy: 0.9555
max_depth=4, CV Accuracy: 0.9539
max_depth=5, CV Accuracy: 0.9508
max_depth=6, CV Accuracy: 0.9487
max_depth=7, CV Accuracy: 0.9466
max_depth=8, CV Accuracy: 0.9425
max_depth=9, CV Accuracy: 0.9373
```

4.8 Zadanie 8

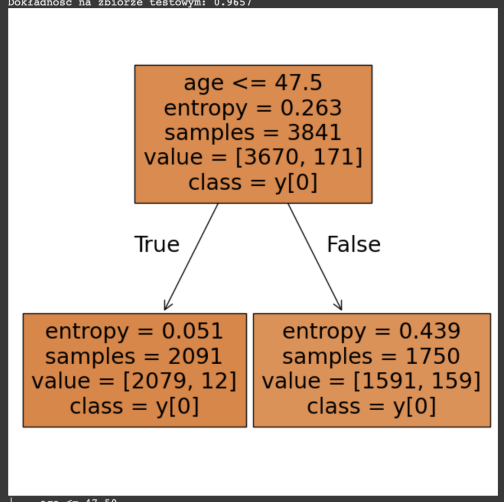
Polecenie: Przeprowadź uczenie drzewa na zbiorze treningowym z optymalną wartością hiperparametru `max_depth` znalezioną w p. 7. Oblicz dokładność klasyfikacji dla danych treningowych i testowych. Narysuj drzewo i przedstaw je w postaci reguł decyzyjnych:

Rozwiązanie:

```python
optimal_depth = 1  #stworzę też dla 2 i 3
dt = DTC(criterion='entropy', max_depth=optimal_depth, random_state=1)
dt.fit(X_train, y_train)

train_accuracy = accuracy_score(y_train, dt.predict(X_train))
test_accuracy = accuracy_score(y_test, dt.predict(X_test))

print(f'Dokładność na zbiorze treningowym: {train_accuracy:.4f}')
print(f'Dokładność na zbiorze testowym: {test_accuracy:.4f}')

fig, ax = plt.subplots(figsize=(8, 8))
plot_tree(dt, feature_names=X.columns, class_names=True, filled=True, ax=ax)
plt.show()

rules = export_text(dt, feature_names=list(X.columns), show_weights=True)
print(rules)
```

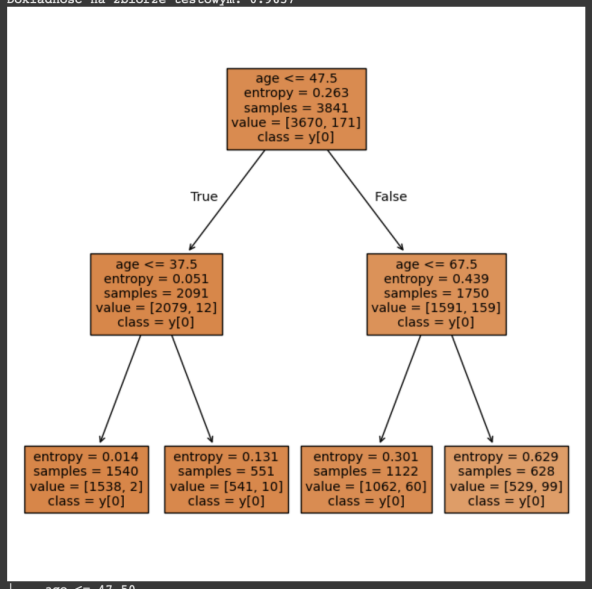
Powyższy kod wykonałam dla wszystkich trzech wyznaczonych optymalnych `max_depth`. Nie jest to konieczne, ale pozwoliło mi zobaczyć, jakie testy, które nie pogarszają dokładności, są dodawane na kolejnych poziomach.

```
Dokładność na zbiorze treningowym: 0.9555
Dokładność na zbiorze testowym: 0.9657
```

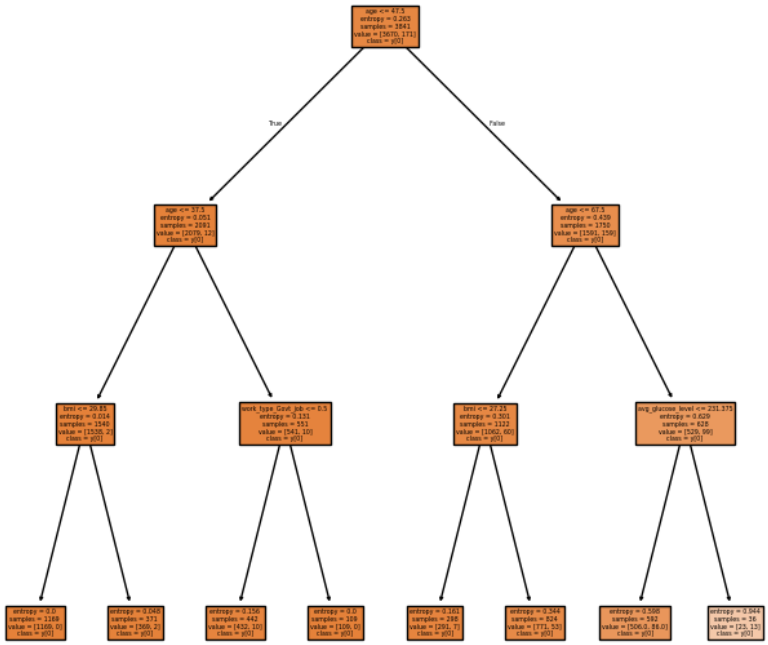



```
|--- age <= 47.50
|   |--- weights: [2079.00, 12.00] class: 0
|--- age >  47.50
|   |--- weights: [1591.00, 159.00] class: 0
```

```
Dokładność na zbiorze treningowym: 0.9555
Dokładność na zbiorze testowym: 0.9657
```




```
|--- age <= 47.50
|   |--- age <= 37.50
|   |   |--- weights: [1538.00, 2.00] class: 0
|   |--- age >  37.50
|   |   |--- weights: [541.00, 10.00] class: 0
|--- age >  47.50
|   |--- age <= 67.50
|   |   |--- weights: [1062.00, 60.00] class: 0
|   |--- age >  67.50
|   |   |--- weights: [529.00, 99.00] class: 0
```



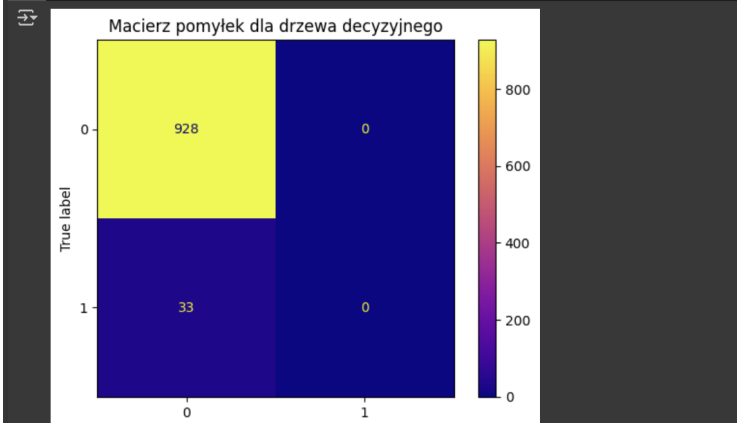
Z drzew wynika, że najważniejsza cecha pozwalająca określić ryzyko udaru to `age`. Wiek jak znaczący w teście na pierwszym i drugim poziomie drzewa. Dopiero przy głębokości równej 3 dochodzą atrybuty `work_type_Govt_job`, `bmi` i `avg_glucose_level`, ale wzięcie ich pod uwagę nie zwiększa dokładności klasyfikacji. Jeszcze bardziej skomplikowane drzewa mają niższą dokładność. Do dalszej pracy wybieram więc najmniej skomplikowane drzewo z tych o najwyższej dokładności.

4.9 Zadanie 9

Polecenie: Zwizualizuj wyniki za pomocą `confusion_matrix`.

Rozwiązanie:

```
cm = confusion_matrix(y_test, dt.predict(X_test), labels=dt.classes_)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=dt.classes_)
disp.plot(cmap="plasma")
plt.title("Macierz pomyłek dla drzewa decyzyjnego")
plt.show()
```



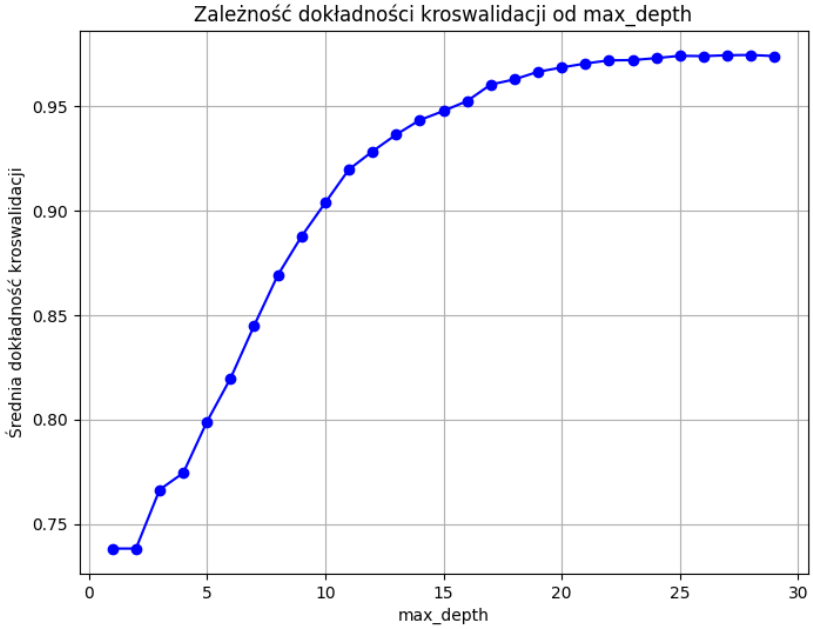
Na macierzy pomyłek widać, że przykłady z klasy 0 są poprawnie sklasyfikowane, ale z klasy 1 – nie. Wynika to z tego, że elementów należących do klasy 0 jest o wiele więcej, więc ich poprawne sklasyfikowanie ma o wiele większy wpływ na dokładność klasyfikatora. Jest to problem niezbalansowanych danych, i próba jego rowzwiązania poprzez oversampling jest przedstawiona w kolejnym zadaniu.

4.10 Zadanie 10

Polecenie: Powtórz p. 7-9 dla danych „rozmnożonych". Czy osiągnięto poprawę wyników? Które atrybuty są najbardziej istotne (występują w początkowych węzłach)?

Rozwiązanie:

```
[85]  ros = RandomOverSampler(random_state=1)
      X_train_resampled, y_train_resampled = ros.fit_resample(X_train, y_train)

[88]  depths = range(1, 30)
      cv_accuracies = []

      for depth in depths:
          dt = DTC(criterion='entropy', max_depth=depth, random_state=1)
          cv_scores = cross_val_score(dt, X_train_resampled, y_train_resampled, cv=5, scoring='accuracy')
          cv_accuracies.append(np.mean(cv_scores))

      plt.figure(figsize=(8, 6))
      plt.plot(depths, cv_accuracies, marker='o', linestyle='-', color='b')
      plt.xlabel('max_depth')
      plt.ylabel('Średnia dokładność kroswalidacji')
      plt.title('Zależność dokładności kroswalidacji od max_depth')
      plt.grid(True)
      plt.show()

      optimal_depth = depths[np.argmax(cv_accuracies)]
      print(f'Optymalna wartość max_depth: {optimal_depth}')
```



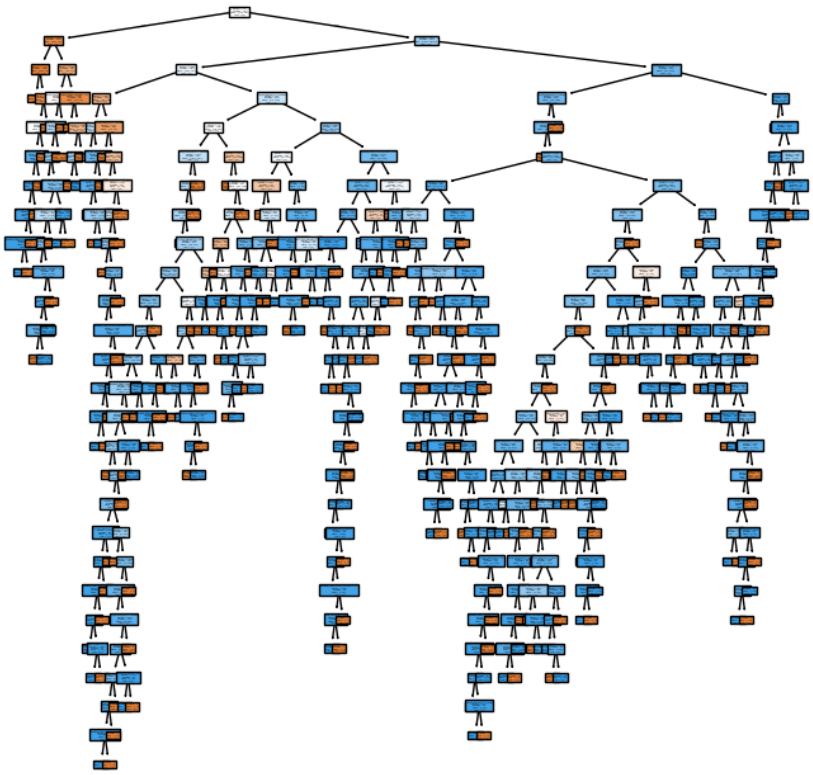
Optymalna wartość `max_depth`: 28. Znacząco większa od `max_depth` na poprzednim, nierozmnożonym zbiorze.

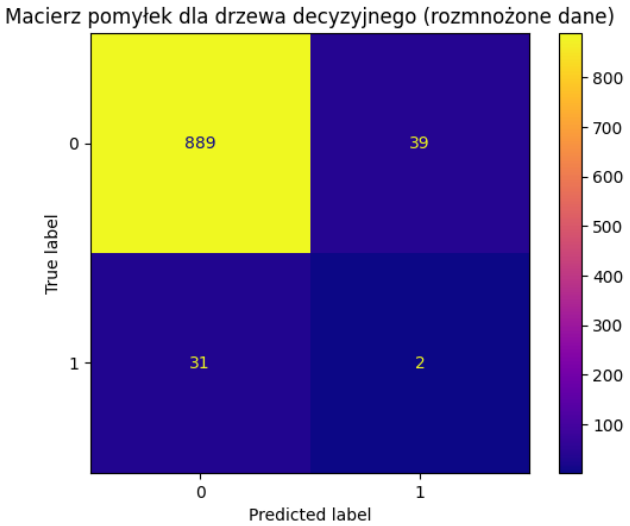
```
[89]
dt = DTC(criterion='entropy', max_depth=optimal_depth, random_state=1)
dt.fit(X_train_resampled, y_train_resampled)

train_accuracy = accuracy_score(y_train_resampled, dt.predict(X_train_resampled))
test_accuracy = accuracy_score(y_test, dt.predict(X_test))

print(f'Dokładność na zbiorze treningowym: {train_accuracy:.4f}')
print(f'Dokładność na zbiorze testowym: {test_accuracy:.4f}')
```

```
Dokładność na zbiorze treningowym: 1.0000
Dokładność na zbiorze testowym: 0.9272
```

```
fig, ax = plt.subplots(figsize=(8, 8))
plot_tree(dt, feature_names=X.columns, class_names=True, filled=True, ax=ax)
plt.show()
```

```
cm = confusion_matrix(y_test, dt.predict(X_test), labels=dt.classes_)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=dt.classes_)
disp.plot(cmap="plasma")
plt.title("Macierz pomyłek dla drzewa decyzyjnego (rozmnożone dane)")
plt.show()
```




Macierz pomyłek dla drzewa decyzyjnego (rozmnożone dane)
889
39
31
2
True label
Predicted label
0
1
0
1
800
700
600
500
400
300
200
100

Macierz pomyłek osiągnięta po zastosowaniu oversamplingu nie pokazuje znaczącej poprawy.

Zwiększona głębokość drzewa i brak znaczącej poprawy dokładnośći na zbiorze testowym wskazują na przeuczenie modelu – oversampling mógł spowodować zbyt dokładne dopasowanie modelu do danych treningowych (szczególnie do sztucznie powiększonej klasy mniejszościowej). Głębokość 28 sugeruje, że drzewo jest bardzo złożone i może próbować wykrywać złożone wzorce, które nie istnieją w danych testowych. Brak poprawy może też wynikać z samych danych - mogą być one zbyt hałaśliwe, albo nie zawierać informacji pozwalających na poprawną klasyfikację klasy mniejszośćiowej. Warto też zauważyć, że mimo, ze drzewo jest o wiele głębsze, to pierwszym testem jest nadal `age`. Potem, podobnie jak w drzewie z głębokością równą 3 z zadania 7., pojawiają sie atrybuty związane z zatrudnieniem i średnim poziomem glukozy.

4.11 Zadanie 11

Polecenie: Używając drzewa regresyjnego dokonaj predykcji wskaźnika `bmi` na podstawie pozostałych atrybutów.

Rozwiązanie:

```
[15]
X = data2.drop(columns=['bmi'])
y = data2['bmi']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

```
max_depths = range(1, 21)
cw = np.zeros(len(max_depths))

for i, depth in enumerate(max_depths):
    dt = DTR(criterion='squared_error', max_depth=depth, random_state=1)
    cw[i] = -cross_val_score(dt, X_train, y_train, cv=10, scoring='neg_mean_squared_error').mean()

optimal_depth = max_depths[np.argmin(cw)]
optimal_depth, min(cw)

(4, np.float64(46.68324377722213))
```

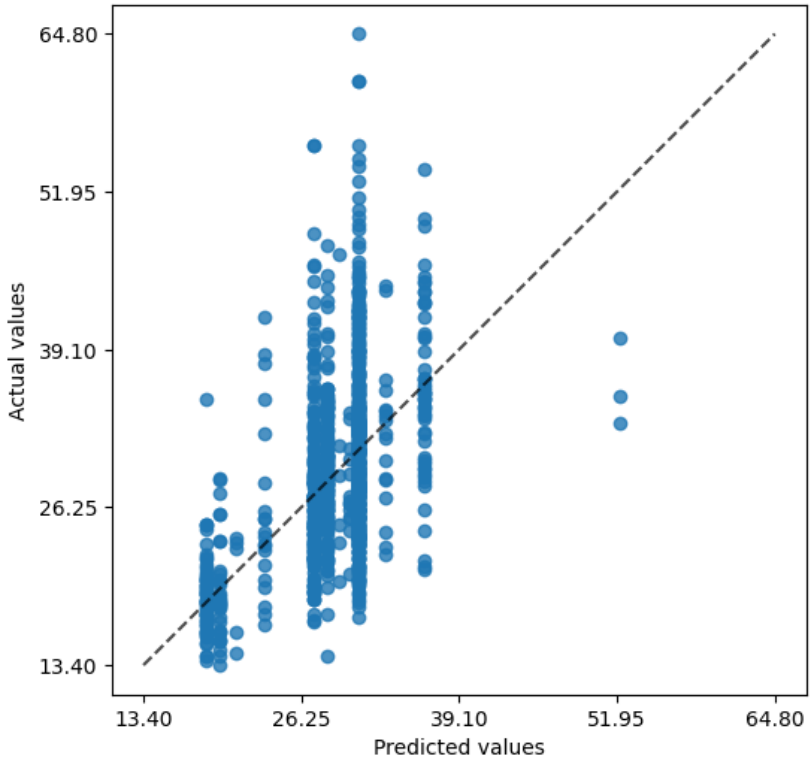
Wyznaczam `optimal_depth`, które minimalizuje błąd (w przypadki klasyfikacji `max_depth` maksymalizowało dokładność). Wynosi ono 4, a minimalny błąd średniokwadratowy wynosi trochę ponad 46.

```
dt = DTR(criterion='squared_error', max_depth=optimal_depth, random_state=1)
dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

ax = subplots(figsize=(6, 6))[1]
PredictionErrorDisplay.from_predictions(
    y_true=y_test,
    y_pred=y_pred,
    kind="actual_vs_predicted",
    ax=ax,
    random_state=0,
)

ax = subplots(figsize=(14, 10))[1]
plot_tree(dt, feature_names=X.columns, filled=True, ax=ax)

print(export_text(dt, feature_names=list(X.columns), show_weights=True))

mse
```

Powyższy kod dokonuje konstrukcji drzewa. Byłam ciekawa, jakie testy zostaną wyznaczone, więc wypisałam je, co zamieszczam poniżej. Pierwsze testy dotyczą tego, czy zatrudnienie to bycie dzieckiem, potem znaczące stają się wiek i średni poziom glukozy.

```
|--- work_type_Children <= 0.50
|   |--- avg_glucose_level <= 182.88
|   |   |--- age <= 29.50
|   |   |   |--- hypertension <= 0.50
|   |   |   |   |--- value: [27.25]
|   |   |   |--- hypertension >  0.50
|   |   |   |   |--- value: [52.19]
|   |   |--- age >  29.50
|   |   |   |--- age <= 66.50
|   |   |   |   |--- value: [30.84]
|   |   |   |--- age >  66.50
|   |   |   |   |--- value: [28.30]
|   |--- avg_glucose_level >  182.88
|   |   |--- age <= 66.50
|   |   |   |--- age <= 33.50
|   |   |   |   |--- value: [29.32]
|   |   |   |--- age >  33.50
|   |   |   |   |--- value: [36.23]
|   |   |--- age >  66.50
|   |   |   |--- age <= 74.50
|   |   |   |   |--- value: [33.11]
|   |   |   |--- age >  74.50
|   |   |   |   |--- value: [30.21]
|--- work_type_Children >  0.50
|   |--- age <= 10.50
|   |   |--- heart_disease <= 0.50
|   |   |   |--- age <= 6.50
|   |   |   |   |--- value: [18.56]
|   |   |   |--- age >  6.50
|   |   |   |   |--- value: [19.62]
|   |   |--- heart_disease >  0.50
|   |   |   |--- value: [29.40]
|   |--- age >  10.50
|   |   |--- avg_glucose_level <= 75.32
|   |   |   |--- stroke <= 0.50
|   |   |   |   |--- value: [20.94]
|   |   |   |--- stroke >  0.50
|   |   |   |   |--- value: [30.90]
|   |   |--- avg_glucose_level >  75.32
|   |   |   |--- avg_glucose_level <= 75.60
|   |   |   |   |--- value: [40.10]
|   |   |   |--- avg_glucose_level >  75.60
|   |   |   |   |--- value: [23.23]

46.256048883454106
```

Poniżej widać wizualizację wyników.



Wysoki bład średniokwadratowy wskazuje na słabą regresję, co potwierdza powyższa wizualizacja. Może to oznaczać, że dane nie zawierają w sobie infomacji pozwalających na przeprowadzenie regresji `bmi`.

4.12 Zadanie 12

Polecenie: Użyj modelu lasu losowego (`RandomForestRegressor`) do rozwiązania problemu regresyjnego postawionego w p. 11. Przyjmij optymalną wartość `max_depth` z p. 11 i domyślne wartości pozostałych hiperparametrów z wyjątkiem `max_features`. W kroswalidacji znajdź optymalną wartość hiperparametru `max_features`.

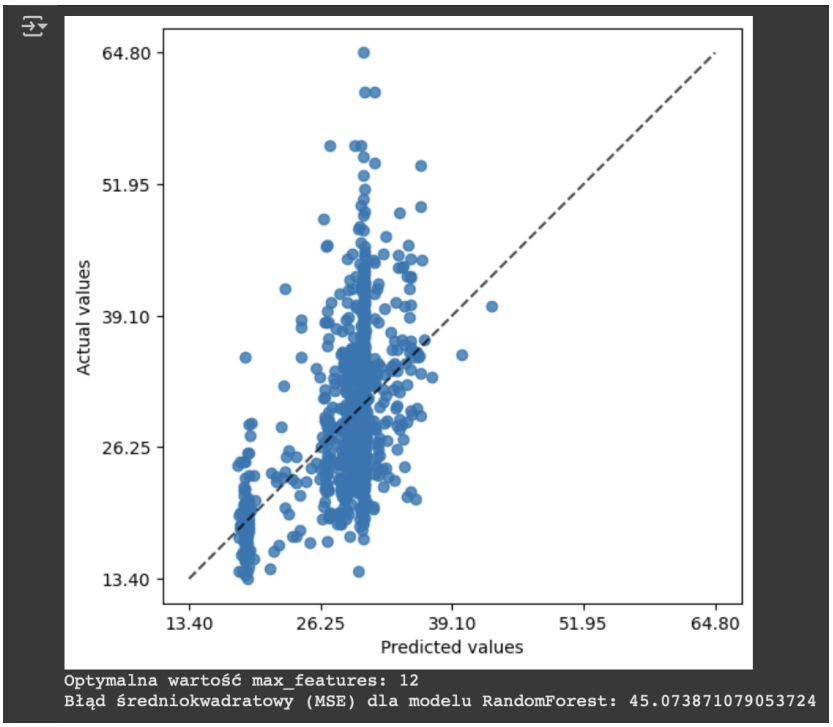
Rozwiązanie:

```python
max_features_range = range(1, X_train.shape[1] + 1)

cv_results = []

for max_features in max_features_range:
    rf = RandomForestRegressor(
        criterion='squared_error',
        max_depth=optimal_depth,
        max_features=max_features,
        random_state=1
    )
    scores = cross_val_score(rf, X_train, y_train, cv=10, scoring='neg_mean_squared_error')
    cv_results.append(-scores.mean())

optimal_max_features = max_features_range[np.argmin(cv_results)]

print(f"Optymalna wartość max_features: {optimal_max_features}")
```

```
Optymalna wartość max_features: 12
```

Wyznaczam wartość `max_features`, która określa maksymalną liczbę cech, które są branych pod uwagę przy każdym podziale drzewa. Jej optymalna wartość wychodzi równa 12, czyli poniżej stworzony model rozważa wszystkie 12 dostępnych cech podczas budowy drzewa. Może to oznaczać, że wszystkie cechy są istotne, lub wręcz przeciwnie – występuje potrzeba lepszego doboru cech.

```python
rf_optimal = RandomForestRegressor(
    criterion='squared_error',
    max_depth=optimal_depth,
    max_features=optimal_max_features,
    random_state=1
)
rf_optimal.fit(X_train, y_train)

y_pred_rf = rf_optimal.predict(X_test)
mse_rf = mean_squared_error(y_test, y_pred_rf)

fig, ax = plt.subplots(figsize=(6, 6))
PredictionErrorDisplay.from_predictions(
    y_true=y_test,
    y_pred=y_pred_rf,
    kind="actual_vs_predicted",
    ax=ax,
    random_state=0,
)
plt.show()

print(f"Optymalna wartość max_features: {optimal_max_features}")
print(f"Błąd średniokwadratowy (MSE) dla modelu RandomForest: {mse_rf}")
```



Tak jak w poprzednim zadaniu, przy oryginalnym, jednym drzewie regresji, dostaję wysoki MSE (błąd średniokwadratowy). Wykres potwiersza przypuszczenie, że regresja nie daje satysfakcjonujących rezultatów. W połączeniu z wysokim `max_features` wnioskuję, że dane nie pozwalają na regresję wysokiej jakości.

4.13 Zadanie 13

Polecenie: Użyj modelu boostingowego opartego na drzewach regresyjnych (`GradientBoostingRegressor`) do rozwiązania problemu regresyjnego postawionego w p. 11. Przyjmij optymalną wartość `max_depth` z p. 11 i domyślne wartości pozostałych hiperparametrów z wyjątkiem `learning_rate` (odpowiednik parametru zawężania λ). W kroswalidacji znajdź optymalną wartość hiperparametru `learning_rate` w zakresie od 0 do 0.1. Porównaj wyniki otrzymane w p. 11-13.

Rozwiązanie:

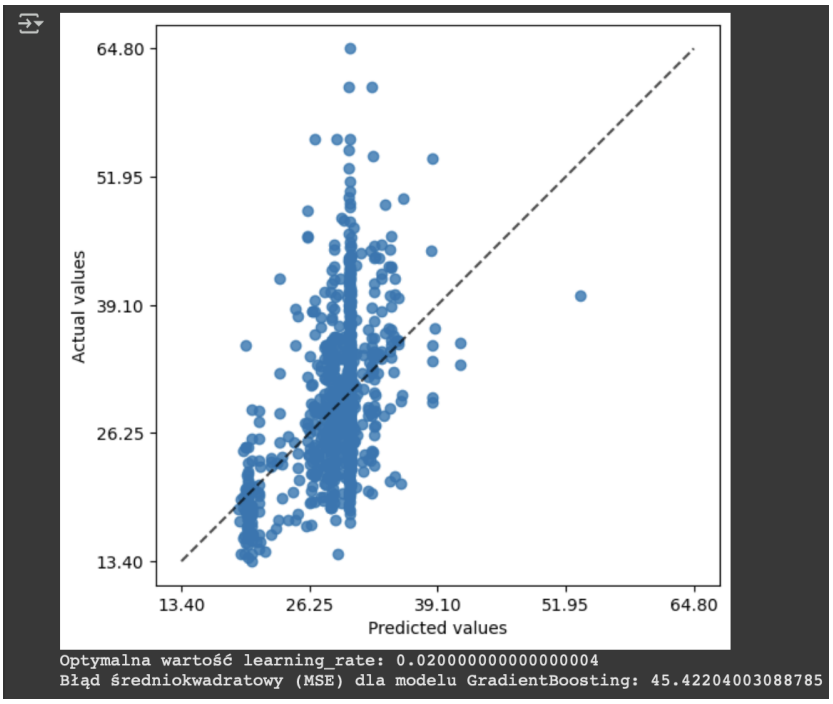
```
[24]  from sklearn.ensemble import GradientBoostingRegressor
```

```python
learning_rate_range = np.linspace(0.01, 0.1, 10)

cv_results = []

for learning_rate in learning_rate_range:
    gb = GradientBoostingRegressor(
        max_depth=optimal_depth,
        learning_rate=learning_rate,
        random_state=1
    )
    scores = cross_val_score(gb, X_train, y_train, cv=10, scoring='neg_mean_squared_error')
    cv_results.append(-scores.mean())

optimal_learning_rate = learning_rate_range[np.argmin(cv_results)]

print(f"Optymalna wartość learning_rate: {optimal_learning_rate}")
```

```
Optymalna wartość learning_rate: 0.020000000000000004
```

Parametr `learning_rate` określa wielkość kroku, z jaką każde kolejne drzewo decyzyjne w sekwencji koryguje błędy poprzednich drzew. Mniejsze wartości `learning_rate` powodują, że model uczy się wolniej, co może wymagać większej liczby iteracji do osiągnięcia optymalnego wyniku, ale często prowadzi do lepszego dopasowania do danych i mniejszego ryzyka przeuczenia. Większe wartości `learning_rate` przyspieszają proces uczenia, ale zwiększają ryzyko przeuczenia się modelu. Wyznaczona wartość 0.02 oznacza, że korekty są stosunkowo małe.

```python
gb_optimal = GradientBoostingRegressor(
    max_depth=optimal_depth,
    learning_rate=optimal_learning_rate,
    random_state=1
)
gb_optimal.fit(X_train, y_train)

y_pred_gb = gb_optimal.predict(X_test)
mse_gb = mean_squared_error(y_test, y_pred_gb)

fig, ax = plt.subplots(figsize=(6, 6))
PredictionErrorDisplay.from_predictions(
    y_true=y_test,
    y_pred=y_pred_gb,
    kind="actual_vs_predicted",
    ax=ax,
    random_state=0,
)
plt.show()

print(f"Optymalna wartość learning_rate: {optimal_learning_rate}")
print(f"Błąd średniokwadratowy (MSE) dla modelu GradientBoosting: {mse_gb}")
```



Na wykresie widać to, na co wskazuje wysokie MSE, czyli słabe wyniki regresji. Wizualne porównanie wykresów z punktów 11-13 pozwala na wstępne potwierdzenie teorii, że drzewa regresyjne nie sprawdzają się dobrze na moich danych przy określaniu `bmi`.

```
1 print("Porównanie wyników:")
2 print(f"DecisionTreeRegressor MSE: {mse}")
3 print(f"RandomForestRegressor MSE: {mse_rf}")
4 print(f"GradientBoostingRegressor MSE: {mse_gb}")
```

```
Porównanie wyników:
DecisionTreeRegressor MSE: 46.256048883454106
RandomForestRegressor MSE: 45.073871079053724
GradientBoostingRegressor MSE: 45.42204003088785
```

Wypisane powyżej błędy średniokwadratowe, wszystkie około wartości 45 i 46, oznaczają, że modele nie radzą sobie dobrze w przewidywaniu wartości BMI (błąd średniokwadratowy na poziomie 45 oznacza, że przewidywania są w przybliżeniu w odległości pierwiastek z 45, czyli ok.6.7, od rzeczywistych wartości.

5 Wnioski końcowe

W ramach tego ćwiczenia zrealizowałam szereg zadań, których celem było zbudowanie i ocenienie klasyfikatora opartego na drzewach decyzyjnych, a także przeprowadzenie regresji wskaźnika `bmi`.
W kwestii ryzyka udaru poruszanego na początku dało się zauważyć, że że najważniejszym atrybutem określającym ryzyko udaru okazał się wiek pacjenta. Wiek był pierwszym testem w wielu drzewach decyzyjnych, co pozwolało sugerować, że jest on silnym predyktorem zdrowotnych zagrożeń, ale dane były niezbalansowane, co wpłynęło na dokładność klasyfikatora. Klasa większościowa (brak udaru) była znacznie lepiej klasyfikowana niż klasa mniejszościowa (udar). Technika oversamplingu nie przyniosła znaczącej poprawy, co może wskazywać na konieczność zastosowania innych metod balansowania danych lub ulepszenia jakości danych.
Próby przewidywania `bmi` za pomocą drzew regresyjnych nie dały satysfakcjonujących rezultatów. Wysokie wartości błędu średniokwadratowego (MSE) pokazały, że modele nie radzą sobie z przewidywaniem tej zmiennej, co sugeruje, że dane nie zawierają wystarczających informacji do dokładnej regresji `bmi`. Modele zespołowe, takie jak lasy losowe i gradient boosting, które w teorii mogły poprawić jakość predykcji, okazały się nieskuteczne, co potwierdziło moje podejrzenia o jakości analizowanych danych.
Wykonanie omówionych w tym sprawozdaniu ćwiczeń pozwoliło mi zauważyć zalety i wady drzew decyzyjnych – są one narzędziem przydatnym do szybkiej eksploracji danych i czytelnej, łatwej do zinterpretowania identyfikacji ważnych atrybutów, ale ich zdolność do dokładnej klasyfikacji i regresji może być ograniczona w przypadku danych o wysokiej nierównowadze klas czy dużej ilości szumów.